

ICS 期末复习 2025

1. 数据类型的转换:

- ① 浮点数的表示
- ② 整数的加减 (标志位变化)
- ③ 数据的对齐 (结构体)

2. 汇编指令:

- ① 压栈 (关注局部变量位置)
- ② 函数建立栈帧的几种方式
- ③ 64 位和 32 位 传递参数的区别

3. 链接:

- ① 重定位: 位置, 偏移量, 符号, 重定位类型.
(容易漏掉作为参数的字符串). `rodata`

5. cache: ④ 分页 ⑤ "段" 存化但被简化 (地址转换)

- ① 组相联
- ② 地址分配
- ③ 命中率

6. 异常中断. I/O

② 挑出 400 ~ 1000 个最能表述 hello 程序执行过程的字.

③ 描述 printf 执行过程. (中断 & I/O)

④ 概念 & 响应过程. (异常)

★ 异常处理过程:

1. 检测异常 (略)

2. 确定检测到的异常类型号为 i , 从 IDTR 指向的 IDT 中取出第 i 个表项 IDT_i .

3. 检查特权级 (略) P_{342}

4. 准备阶段:

1. 在当前栈中保存 EFLAGS, CS, EIP 中的内容

2. 将 IDT_i 中的段选择符装进 CS, IDT_i 的偏移地址装入 EIP,

5. 处理阶段:

1. Linux 采用信号机制处理异常, 在对应类型号的处理程序 _____ 中, 会发送 _____ 信号给用户进程, 随后转引用户进程执行.

2. 用户进程接受信号后, 转引到对应信号处理程序执行.

6. 恢复阶段:

1. 从内核栈中弹出 EIP, CS, EFLAGS, 恢复断点和程序状态.

2. 检查特权 (略) P342

★ 系统调用: (printf 执行过程)

1. 首次调用 (动态链接)

① 首次执行 printf 时, 通过执行延迟绑定代码, 确定 printf 在内存中的首地址.

② 将找到的首地址填入对应 GOT 表中 (后续再调用时, 将直接跳转到该地址执行)

2. 参数准备

printf 内部最终会调用 write, 但在执行具体调用前, 处理器在用户态完成参数准备.

① 将 write 的系统调用号 4 放入 EAX

② EBX 存放文件描述符, ECX 存放待打印字符串首地址, EDI 存放字符串长度.

若参数超过 6 个, 则将参数块首地址放入寄存器中.

3. 准备阶段

① 执行 `int $0x80` 指令

② 检查特权级 (略)

③ 将用户态的 EFLAGS, CS, EIP 的内容压栈

4. 处理阶段:

- ① CPU 转到系统调用处理程序 `system-call` 的首地址执行.
- ② `system-call` 根据 `EAX` 中的调用号, 跳转到对应服务例程 `system-write`.
- ③ 在内核中依次通过 设备类层, 设备驱动程序, 最终操作 I/O 外设完成字符显示.

5. 恢复阶段:

- ① 通过 `iret` 指令返回用户态.
- ② 返回结果存放在 `EAX` 中, >0 为成功, 负数表示出错码.
- ③ 程序回到用户空间下条指令继续执行.

数据类型转换:

数据在计算机中的机器码相同, 只是不同数据类型的解释不同.

1. 运算整形提升原则:

$$a = b + c$$

① 立即数、优先级小于 `int` 的数在参与运算时会被转为 `int`

(注意, 这里无符号数也被解释为 `int`) `unsigned short` $\rightarrow$ `unsigned`

② 进行运算时, 先向上对齐类型, 再计算.

③ 赋值时, 判断

- 目标类型是否更窄,
- 目标类型是否带符号.

1) 低有 $\rightarrow$ 高有 $\checkmark$

2) 低有 $\rightarrow$ 高无 正数不变, 负数取模.

3) 低无 $\rightarrow$ 高有 $\times$

4) 低无 $\rightarrow$ 高无 $\checkmark$

5) 高有 $\rightarrow$ 低无 取模

6) 高无 $\rightarrow$ 低无 取模

7) 高有 $\rightarrow$ 低有 $\times$

8) 高无 $\rightarrow$ 低有 $\times$

标志位

SF, OF: 解释

① 带符号数: OF

① 负-负=负 SF=1 OF=0 <

② 无符号数: CF

② 负-负=正 SF=0 OF=0 >

ja 无符号 CF=0 ZF=0

③ 负-正=负 SF=1 OF=0 <

jb 无符号 CF=1 ZF=0

④ 负-正=正 SF=0 OF=1 <

jg 带符号 SF=OF ZF=0

⑤ 正-负=正 SF=0 OF=0 >

jl 带符号 SF!=OF

⑥ 正-负=负 SF=1 OF=1 >

⑦ 正-正=负 SF=1 OF=0 <

⑧ 正-正=正 SF=0 OF=0 >

(x) 更新逻辑:

OF: 负+负=正

正+正=负

CF: sub @ cout

CF: { 加法: dst, src 对应无符号数运算是否有进位

减法: dst, src 对应无符号数是否有 $dst < src$

OF: { 加法: dst, src 对应带符号数是否正+正=负

① 运算类:

add sub adc sbb neg: OF CF SF ZF
(imp)

inc dec : 半更新CF

② 逻辑类

mul, imul 不看 SF, ZF

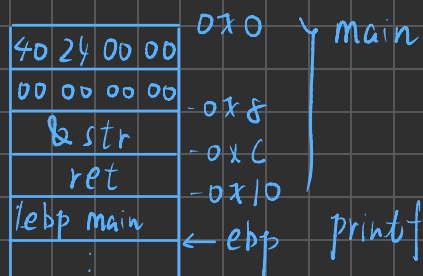
and or xor ZF SF 半 CF, OF 清零
(test)

过程调用:

1. 画栈帧: 根据地址宽度调整大小, P140

IA32:

(行内使用大端, 行间使用小端)



一定按照汇编代码一步步画!

2. 建立栈帧的几种方式:

① pushl %ebp

movl %esp, %ebp

subl \$N, %esp

leave → movl %ebp, %esp

ret → popl %ebp

popl %eip

jmp *%eip

② subl \$N, %esp

addl \$N, %esp

ret

3. 寄存器

IA-32

- ① { 调用者保存寄存器: `eax ecx edx`
被调用者保存寄存器: `ebx esi edi`

② 用栈传参, 栈中参数按 4 字节对齐.

通常用 `ebp` 作为 `base` 访问局部变量、参数.

x86-64

- ① 调用者保存寄存器: `r10, r11`

被调用者保存寄存器: `rbx, rbp, r12, r13, r14, r15`

- ② 当参数小于等于 6 个时, 依次放在 `rdi, rsi, rdx, rcx, r8, r9`

大于 6 个时, 用栈传参, 按 8 字节对齐.

通常用 `esp` 作为 `base` 访问局部变量、参数.

P181. 26

$\text{sizeof}(\text{int}) + \text{sizeof}(x) + \text{offset}$

$$R[ecx] \leftarrow i \quad \leftarrow \text{sptr} + \underset{\uparrow}{200} = 4 + 28 \times 7 \quad \text{LEN} = 7$$

$$R[ecx] \leftarrow \text{sptr}$$

$$R[ebx] \leftarrow i * 28$$

$$R[edx] \leftarrow i * 8 \quad \leftarrow i * 7 \quad \leftarrow \frac{M[\text{sptr} + 4 + \underset{\uparrow}{i * 28}]}{\text{sptr} \rightarrow x[i]} + i * 7$$

$\text{sizeof}(\text{ln_struct})$

$$M[\text{sptr} + 8 + (\text{sptr} \rightarrow x[i] * 4) + (i * 7) * 4]$$

$$= M\left[\frac{\text{sptr} + 4 + \underset{\text{sptr} \rightarrow x[i] \rightarrow a}{i * 28} + 4}{\text{idx}} + \frac{(\text{sptr} \rightarrow x[i]) * 4}{\underset{\uparrow}{\text{sizeof}(\text{type_a})}}\right]$$

$\downarrow$
 $\text{xptr} \rightarrow a[\text{xptr} \rightarrow \text{idx}]$

$\therefore \text{LEN} = 7;$

typedef struct {

int idx

unsigned a[6];

} line_struct;

链接:

1. 在 `m.o` 符号表中的符号类型:

全局符号: 由 `m.o` 定义并被其它模块引用的全局变量.

外部符号: 由别的模块定义并被 `m.o` 引用的全局变量.

本地符号: "static"

2. 符号: 非静态局部变量不是符号. P200

强符号: 已初始化全局变量 `int x = 10;`

函数 `int foo();`

★ `bss` 节中初始化为 0 的全局变量 `int x = 0;`

COMMON: 未初始化全局变量.

弱符号: (otherwise)

3. 重定位

① R_386_PC32

跳转目标地址 = PC + offset (重定位值)

offset = 跳转目标地址 - PC

跳转目标地址 = 跳转目标符号的定义在可执行文件中的地址

PC = 下一条指令地址

= 目标符号引用在可执行文件中的地址 + 地址长度

= 当前代码段在可执行文件中的地址

目标符号引用⁺在当前代码段中偏移⁺
地址长度

= addr(.text) + r_offset + (-init)

[△]
在可重定位文件中填入的初始值

② R_386_32

最后生成可执行文件中地址的要加上偏移量。

Cache:

1. 映射方式:

① 直接映射: 模映射

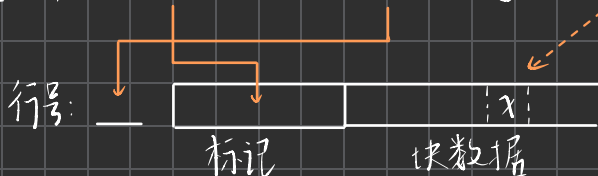
cache 行号 = (主存块号) mod (cache 行数)

cache 有 2^c 行, 主存块有 2^m 块;

cache-行大小 = 主存-块大小 = $2^b B$;

$M[addr] = x$

$addr[m+b-1, c+b] : addr[c+b-1, b] : addr[b-1, 0]$



cache 容量: $(1 \text{ 有效位} + t \text{ 标记} + 2^b \text{ 大小}) \times 2^c \text{ 行数}$

P256

cache-行 $2^9 B$ - 共 2^c 行 $c=4$ $2^3 B$

主存 - 块 $2^9 B$ - 共 2^m 块 $m=11$ $2^{20} B$

主存地址位数 $n=20$

标记位数 $t=7$

行号位数 $c=4$

块内地址位数: 9

主存块号		
标记位数	cache 行号	块(行)内地址
0000 001	0010	0 0000 1100

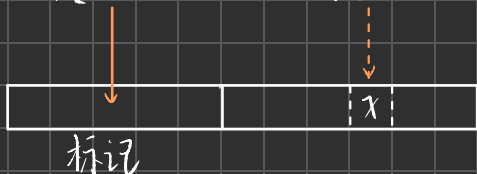
0000 0010 0100 0000 1100 B

② 全相联方式:

$$M[addr] = x$$



$$addr[m-1, b] : addr[b-1, 0]$$



P258

cache - 行 $2^9 B$ - 共 2^9 行 $2^9 B$

主存 - 块 $2^9 B$ - 共 2^{11} 块 $2^{20} B$

主存地址位数 $n = 20$ 位

主存标记位数 $t = 11$ 位

块内地址位数 $b = 9$ 位

标记 & 主存块号 块内地址

0000	0010	010	0	0000	1100
------	------	-----	---	------	------

0000 0010 0100 0000 1100 B

③ 组相联方式 (组间直接映射, 组内全相连)

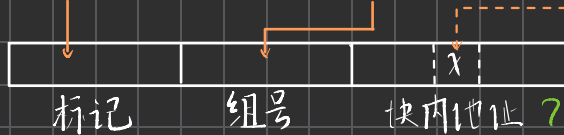
$$\text{cache 组号} = (\text{主存块号}) \bmod (\text{cache 组数})$$

标记 2^t , 组数 2^g , - 组 2^s 行,

$$M[addr] = x$$



$$addr[t+q+b-1, q+b] : addr[q+b-1, b] : addr[b-1, 0]$$



P259

cache 一行 $2^9 B$ 一共 2^4 行 $2^{13} B$

- 组 2^1 行 一共 2^3 组

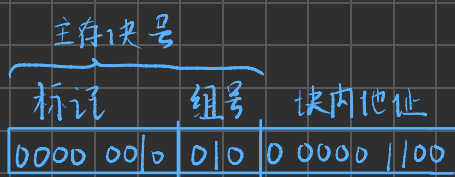
主存 一块 $2^9 B$ 一共 2^{11} 块 $2^{20} B$

主存地址位数 $n = 20$

标记位数 $t = 8$

组号位数 $g = 3$

块内地址位数 $b = 9$



0000 0010 0100 0000 1100 B

2. 替换算法.

① 最近最少用算法: LRU

- 为每一个 cache 块设置一个“计数器”，记录每个 cache 块多久未被访问。cache 满后替换“计数器”最大的块。
- 算法步骤:

invariant: “计数器”维护的是一个 $[0, \dots, n-1]$ 的全序序列

- ① 命中时，所命中的行清零，比其低的行加 1，比其高的行不变。
- ② 未命中且有空闲行时，新装入的行置零，其余非空闲行加 1。
- ③ 未命中且无空闲行时，替换最高行并置零，其余行加 1。

3. 写策略

① 通写法: 写命中时, 同时写 cache 和主存.

写缺失时, 有以下两种解决方法.

1. 写分配法: 更新主存后, 为更新后的主存块分配一个 cache 行.

2. 非写分配法: 更新主存后不更新 cache.

② 回写法: 若写命中, 则只将内容写入 cache 而非主存,

并设置修改位为 1

若写不命中, 则“写分配”

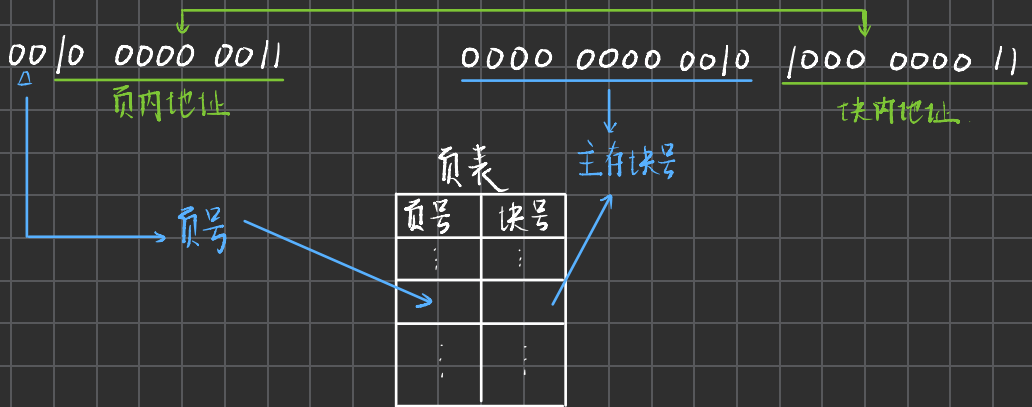
分页机制:

页表主存: 逻辑划分, cache 主存: 物理划分.

物理地址 $\xrightarrow[\text{(页表)}]{\text{分页}}$ 逻辑地址 [虚拟地址]
(主存中的地址) (程序员看到的地址)

e.g. 某程序 4 KB

主存 4 MB



页表存放在主存块中, 故每次地址转换都要访问.

故可仿照 cache 原理, 设计 快表 TLB, 将经常访表的页表项

从 页表 (主存) 搬入 TLB (cache) 中.

全相联或组相联

2023 (A) 六. 6 D

TLB - 组 2^7 行 - 共 2^6 组 2^7 行

页表 2^{20} 块

20位 16位 4位
主存块号 = 标记 : 组号

地址转换:

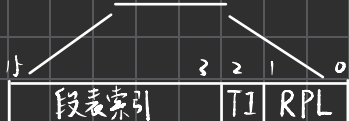
1. 逻辑地址 $\rightarrow$ 线性地址

e.g. `movl %ebx, %eax`



这条指令属于代码段的一部分

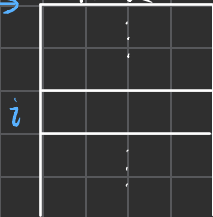
逻辑地址: $R[CS]$: 有效地址



TI=0 LDT
TI=1 GDT
(这里以 TI=1 为例)

段表

GDTR $\rightarrow$



64位

Base

DPL 访问该段所需权限

$DPL \geq \max(CPL, RPL)$

$G = \begin{cases} 0 & \text{以字节为单位} \\ 1 & \text{以页为单位} \end{cases}$

$L_{20\text{位}}$ 界限



线性地址

访问以后将对应段表项放入 cache 中

后续直接从 cache 中读取

★ 地址转换 (这是从一道题中摘出来的, 要配添一些逻辑)

8048577: mov 0x804a064(, %eax, 8), %eax

用300字描述该指令执行过程。

- ① 根据指令地址中高20位 0x 08048 访问 TLB, 得到指令物理地址。
- ② CPU 根据物理地址访问 cache 命中, 从 cache 中取指令。
- ③ 指令译码
- ④ 计算源操作数有效地址: $0x804a064 + R[eax] * 8$
- ⑤ 根据有效地址高20位访问 TLB
- ⑥ 若命中, 则从 TLB 中取出主存块号, 与有效地址低12位拼成物理地址。
若缺失, 则去慢表中找到对应页表项, 若有效位为1, 则取出主存块号与有效地址低12位拼成物理地址。若有效位为0, 则缺页异常, 转入内核处理。
- ⑦ 根据物理地址去 cache 中查询。
若命中, 则取出相应数据。若缺失, 则 CPU 到主存中取出主存块后更新 cache (LRU 替换, 有效位置, 标记更新) 后取出数据。
- ⑧ 将数据存入 EAX 中。